

DAT32-1

Intro to Machine Learning

[Module page](#)

Version of 13 September 2026

About this textbook

This year, for the first time, every Bachelor course at Albert School has its own textbook, thanks to your teachers and AI.

- The textbook is a help for working on the course at home, after the class.
- The course as taught in class remains the reference, and prevails over the textbook wherever the two differ.
- This is a first version. Help us improve it by sharing your feedback with your teacher.

Contents

Preface	4
1. What machine learning is: taxonomy and workflow	5
2. Core vocabulary, built by hand	7
3. Simple linear regression from first principles	10
4. Baselines and evaluation metrics	14
4.1. The baseline: the minimum a model must beat	14
4.2. Metrics: how wrong, in what units	14
5. Gradient descent	16
6. Underfitting and overfitting	19
6.1. The validation curve: which complexity?	19
6.2. The learning curve: does more data help?	20
7. Regularization	23
8. Cross-validation	26
9. Hyperparameters	28
10. From simple to multiple regression	30
11. Reproducible workflows: scikit-learn pipelines	33

Preface

You arrive here already able to handle data. You can load, clean, filter, group and reshape tabular data with pandas, and reason about the cost of an operation. You can turn a business question into a SQL query and read its answer. You can make code reproducible and testable with Git, virtual environments, classes and pytest. From mathematics you bring the mean and variance, the idea of a probability distribution, the derivative of a function, and matrix multiplication. This book assumes all of that and builds on it; it does not re-teach it.

What you do **not** yet have is a model — a thing that takes data it has never seen and predicts a number. That is what machine learning adds, and this course teaches it in an unusual way: through **one** model, linear regression, followed all the way from a raw table to a tuned, validated, reproducible pipeline. Every core idea in machine learning — loss functions, optimisation, overfitting, baselines, evaluation metrics, regularization, cross-validation, hyperparameters — is introduced at the moment it first becomes necessary in that single workflow, not as an entry in a glossary.

This is deliberate. Spend all of your study on one model understood completely and you do not learn “linear regression”; you learn the **shape** of every machine-learning project. When you later meet logistic regression, decision trees or neural networks, you will not be learning a new paradigm — you will be substituting a new model into a cycle you already own.

A word on method. Every idea arrives as a concrete example first and the general statement second, and every technique is **implemented**, not just described — this is a course where you write the code. We keep one running problem throughout — **predicting the monthly rent of an apartment from its characteristics** — mostly on a single dataset of two hundred apartments that we generate in code so that every number in this book is one you can reproduce exactly on your own machine. Two ideas — the curvature that produces overfitting, and the effect of dataset size — need data shaped differently from that main sample, so where they arise we generate small purpose-built rent samples the same way, each introduced where it is used. Run the snippets as you read; the printed outputs are what you should see. We use numpy for the from-scratch work and scikit-learn once the ideas are in place.

1. What machine learning is: taxonomy and workflow

Suppose you manage a rental portfolio and want to predict the monthly rent of a flat before it goes on the market. You could try to write the rule by hand — “start at €400, add €12 per square metre, add €150 if it has a balcony...” — but you would be guessing the numbers. Machine learning instead **learns** those numbers from past rentals: you show an algorithm many (apartment, rent) pairs and it fits a model that reproduces them as closely as possible, then applies that model to new apartments.

Definition 1. Machine learning builds a model that maps inputs to an output by fitting it to data, rather than by being programmed with explicit hand-written rules. The fitted model is then used to predict the output for new, unseen inputs.

Before zooming in on our one model, here is the map of the territory — just enough to know where regression sits. Start with three tasks a data team might be handed.

Example — three tasks, three shapes. “Predict tomorrow’s electricity demand in megawatts”: each past day comes with the demand that actually occurred, and the answer to predict is a number. “Will this customer churn next month, yes or no?”: each past customer comes with a known outcome too, but now the answer is one of two categories. “Split our 50 000 customers into marketing segments we have not defined in advance”: here no example comes with a right answer at all — the task is to find structure no one labelled. Three tasks, and already three different shapes.

Those shapes are the whole map, and each has a name.

Definition 2. Supervised learning fits a model on data where every example carries a known target value (the electricity demand, the churn outcome). **Unsupervised learning** works on data with no target and instead discovers structure — groups, directions — in the inputs alone (the customer segments).

Definition 3. Within supervised learning, **regression** predicts a continuous numeric target (a rent, a temperature, a demand in megawatts) and **classification** predicts a discrete category (churn yes / no, which of five products). **Clustering** is the archetypal unsupervised task: it partitions examples into groups without any target.

The task type is decided by the target, not by the field or the difficulty. Ask: **is there a target to predict, and is it a number or a category?** A number to predict is regression; a category is classification; no target, “find the natural groups,” is clustering. Rent prediction is regression: the target, euros per month, is continuous.

Whatever the model, a supervised-learning project follows the same cycle. The rest of this book **is** this cycle, walked once on linear regression.

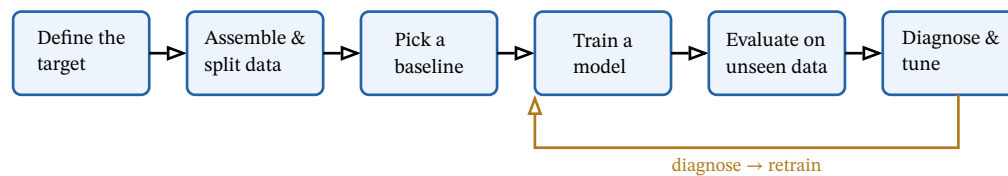


Figure 1: The supervised machine-learning workflow. It is model-independent: this course executes every step on linear regression, but the same loop drives every model you will meet later. The orange arrow is the part that matters most in practice — you rarely train once; you train, diagnose, and retrain.

Exercises

Throughout the exercises, use the running rent dataset (`make_rentals`) with the same fixed splits unless a problem says otherwise, so your numbers match those quoted here; each exercise ends with a check you can run.

1.1. In one sentence each, say what distinguishes supervised from unsupervised learning, and regression from classification. Then invent one task of your own for each of the three types. *Check: your regression task has a numeric target, your classification task a categorical one, and your clustering task no target at all.*

1.2. For five stated business problems, name each as regression, classification, or clustering, and say whether it is supervised or unsupervised. Justify each by identifying the target (or its absence). *Check: every “no target” problem must be unsupervised, and every category target must be classification.*

2. Core vocabulary, built by hand

Rather than list definitions, we build the vocabulary by constructing the dataset our model will use. So that every later number is reproducible, we **generate** the data with a fixed random seed. Run this once; you now have the two hundred apartments this whole book uses.

```
import numpy as np

def make_rentals(n=200, seed=42):
    rng = np.random.default_rng(seed)
    surface = rng.uniform(20, 120, n).round()           # square metres
    rooms = np.clip(np.round(surface/28 + rng.normal(0, 0.4, n)), 1, 5)
    balcony = rng.binomial(1, 0.4, n)                  # 0 or 1
    rent = (250 + 9.5*surface + 55*rooms + 110*balcony
            + rng.normal(0, 70, n)).round()             # euros / month
    return surface, rooms, balcony, rent

surface, rooms, balcony, rent = make_rentals()
```

We built this table from a rule we chose (€9.5 per m², €55 per room, €110 for a balcony, plus random noise) precisely so we can later check whether the model **recovers** it. Real data comes with no such answer key — that is exactly why we must learn to **estimate** the rule and to **measure** how well we did. Here are the first six apartments:

Surface (m ²)	Rooms	Balcony	Rent (€/month)
97	4	no	1504
64	2	yes	992
106	4	no	1494
90	3	no	1397
29	1	no	465
118	5	yes	1666

Each **row** is an example. The columns **surface**, **rooms** and **balcony** are the inputs we predict from; the column **rent** is what we want to predict.

Definition 4. A **feature** is an input variable used to predict the target (surface, rooms, balcony). A **label** is the known target value for a given example (the rent). A supervised dataset is a table of examples, each row pairing a feature vector with its label.

We write one example as a feature vector $x = (x_1, \dots, x_p)$ with label y , and the whole dataset as n pairs $(x^{(i)}, y^{(i)})$ for $i = 1, \dots, n$. Here $n = 200$ and, with all three features, $p = 3$.

Now the subtle part. If we judge the model on the very apartments it was fitted to, we measure memorisation, not prediction. So we **split** the data into parts with distinct jobs.

Definition 5. The **training set** is the data the model is fitted on. The **validation set** is held out from training and used to compare models and tune choices. The **test set** is used exactly once,

at the very end, to estimate performance on unseen data. An example used to choose or tune the model must never also be used to report its final performance.

Two more terms name **how** we hold data out, and they are worth telling apart from the sets above.

Definition 6. A **holdout set** is any set of examples kept aside and never shown to the model during training, so that performance can be measured on data the model has not seen; the **holdout method** is the strategy of estimating performance from a single such split (as opposed to cross-validation, Chapter 8). The validation set and the test set are both holdout sets — “holdout” names the general idea, while “validation” and “test” name the two specific jobs the held-out data does.

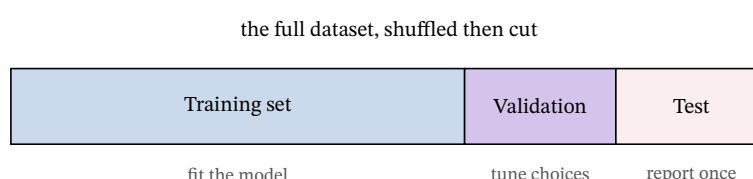


Figure 2: The three-way split. A typical division is 60/20/20, but the exact proportions matter less than the discipline: the test set is kept aside until the very end. Use it any earlier and its score can no longer be trusted.

We use `scikit-learn`’s helper to shuffle and cut. Splitting once, with a fixed `random_state`, gives everyone the same three sets:

```
from sklearn.model_selection import train_test_split

X = np.column_stack([surface, rooms, balcony])          # shape (200, 3)
X_tr, X_tmp, y_tr, y_tmp = train_test_split(X, rent, test_size=0.4, random_state=0)
X_val, X_te, y_val, y_te = train_test_split(X_tmp, y_tmp, test_size=0.5,
                                             random_state=0)
print(len(y_tr), len(y_val), len(y_te))
```

```
120 40 40
```

Pitfall. The single most common mistake in machine learning is to let the test set influence the model — even indirectly, by looking at its score while choosing a setting. A model evaluated on data it has effectively already seen looks far better than it is. Split first, then forget the test set exists until the final line of the project.

Exercises

2.1. From the six-apartment table shown earlier in this chapter, write out the feature vector x and the label y for the first apartment, and state which columns are features and which is the label. *Check: three features (surface, rooms, balcony) and one label (rent); the first apartment’s label is €1504.*

2.2. Load the rent dataset, identify its features and label, and split it 60/20/20 into training, validation and test sets. Explain in one sentence each what the three sets are for and why the test set must stay sealed. *Check: the split sizes are 120 / 40 / 40.*

3. Simple linear regression from first principles

Start with one feature. Plot rent against surface and the points fall almost on a straight line — bigger flats cost more, at a roughly constant rate per square metre. A model that captures exactly this is the simplest useful one.

Definition 7. **Simple linear regression** models the target as a linear function of one feature:

$$\hat{y} = \beta_0 + \beta_1 x,$$

where β_0 is the **intercept** and β_1 is the **slope**. The hat on $\hat{y}$ marks it as the model's prediction, distinct from the true label y .

For any choice of (β_0, β_1) the line misses each point by some vertical gap, the **residual** $y^{(i)} - \hat{y}^{(i)}$. Fitting the model means choosing the line that makes these gaps collectively smallest — but “smallest” needs a precise meaning, and that is the job of a cost function.

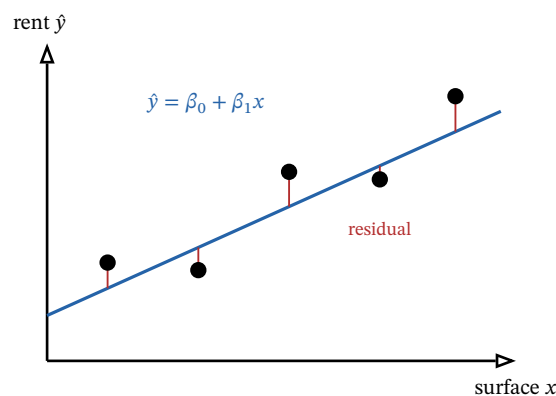


Figure 3: Simple linear regression on the rent data. The blue line is the model; each red segment is a residual, the vertical gap between a real rent (black dot) and the line's prediction. Ordinary least squares chooses the line that minimises the sum of the **squared** residual lengths.

Definition 8. The **residual sum of squares** (RSS) measures total miss as the sum of squared residuals over the training set:

$$\text{RSS}(\beta_0, \beta_1) = \sum_{i=1}^n (y^{(i)} - \beta_0 - \beta_1 x^{(i)})^2.$$

Why **squared** and not, say, absolute? Squaring makes the cost a smooth bowl with a single lowest point, which can be minimised exactly by calculus, and it penalises large misses disproportionately. Setting the two partial derivatives of the RSS to zero and solving gives a closed-form answer.

Definition 9. **Ordinary least squares** (OLS) chooses the coefficients that minimise the RSS. For simple regression the minimiser is available analytically:

$$\beta_1 = \frac{\sum_{i=1}^n (x^{(i)} - \bar{x})(y^{(i)} - \bar{y})}{\sum_{i=1}^n (x^{(i)} - \bar{x})^2}, \quad \beta_0 = \bar{y} - \beta_1 \bar{x},$$

where $\bar{x}$ and $\bar{y}$ are the means of the feature and the label.

These two formulas **are** the algorithm — we can implement OLS in a few lines of numpy, with no library fitting routine, and read off the coefficients.

```
def ols_simple(x, y):
    x_bar, y_bar = x.mean(), y.mean()
    beta1 = ((x - x_bar) * (y - y_bar)).sum() / ((x - x_bar) ** 2).sum()
    beta0 = y_bar - beta1 * x_bar
    return beta0, beta1

x_tr = X_tr[:, 0] # surface column of the training set
beta0, beta1 = ols_simple(x_tr, y_tr)
print(round(beta0, 2), round(beta1, 3))
```

```
264.61 12.0
```

Example — the slope is a familiar quantity. The numerator of β_1 is the sample covariance of x and y (times n), and the denominator is the sample variance of x (times n). So $\beta_1 = \text{Cov}(x, y) / \text{Var}(x)$ — the slope is covariance normalised by the spread of the feature. The mean, variance and covariance you met in statistics are exactly the ingredients of the least-squares line.

Interpreting the coefficients. The slope $\beta_1 = 12.0$ is the predicted change in rent for a one-square-metre increase in surface: each extra square metre adds about €12 to the predicted monthly rent. (We built the data with a €9.5/m² rule, so why 12? Because bigger flats also tend to have more rooms, and with surface as the **only** feature its coefficient also absorbs that room effect — a point we return to in Chapter 10.) The intercept $\beta_0 = 264.61$ is the predicted rent at surface zero — not literally meaningful (no flat has zero area), but necessary to anchor the line’s height.

Pitfall. A regression coefficient is an **association in this dataset**, not a proven cause. “Each square metre adds €12” describes the fitted line; it does not prove that enlarging a specific flat will raise its rent by that amount, because surface is entangled with rooms, location, age and everything else that varies alongside it.

OLS returns a line for **any** data. But if you want its coefficients to support trustworthy **inference** — confidence intervals, significance tests — the data must satisfy modelling assumptions. Rather than meet them as a list, watch them fail, one tier at a time, on the rent data.

Example — the assumptions, seen where they fail. Is the line centred right? We already saw surface’s slope come out €12 where the rule we built in was €9.5. Rooms — left out of the one-feature model — rises with surface, and its effect leaked into the surface coefficient. That leak is a biased estimate, and it is exactly what the **mean-independence** assumption forbids: it asks that whatever the model leaves out average to zero at every surface value, so nothing systematic is left

for the coefficient to absorb. When an omitted variable is entangled with a feature, the estimate is biased — and no quantity of extra data repairs it.

Is the reported precision right? Suppose instead that the rents of large flats scattered far more widely, in euros, than those of small flats — the residual band fanning open as surface grows (Figure 4, right). The line would still sit in the right place, but the “ $\pm$ ” it reports on the slope would be wrong: this is **non-constant variance**. Errors that carried information about one another — whole buildings mispriced together, say — would spoil that reported precision the same way.

Can we read a p -value off it? Even with the line centred and its precision right, turning that precision into an exact confidence interval or significance test from a **small** sample needs the errors to be roughly **normal**. With many flats the central limit theorem makes that reading approximately valid regardless.

Each question isolates one tier; the box below states all three in general.

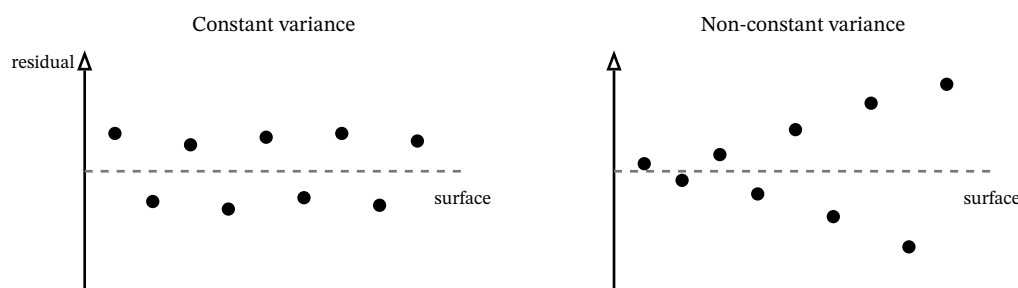


Figure 4: Residuals plotted against the feature. Left: a constant-width band — the spread of the misses does not depend on surface (homoscedasticity), the condition OLS’s standard-error formula assumes. Right: a band that fans open as surface grows (heteroscedasticity) — the line is still centred on zero, but the precision OLS reports for its coefficients can no longer be trusted.

Definition 10. For OLS, the estimated coefficients are **unbiased** (correct on average) when the relationship is **linear in the coefficients** and the errors are **mean-independent of the features** — for every value of the features their average is zero, $\mathbb{E}[\varepsilon | x] = 0$, the condition called **strict exogeneity**. This is stronger than the errors being merely **uncorrelated** with the features: errors that are uncorrelated but not mean-independent still leave OLS correct only as the sample grows large (**consistency**), not unbiased in every finite sample. Two further assumptions make the usual **standard-error formula** correct and the estimator the most precise of its kind (the Gauss–Markov result): the errors have **constant variance** (homoscedasticity) and are **uncorrelated with one another** (no autocorrelation). For the **t- and F-based inference** built on those standard errors — confidence intervals, significance tests — to be **exactly** valid in a finite sample, one assumption more is required: the errors are **normally distributed**. Without normality that inference is only **approximate**, justified in large samples by the central limit theorem.

The three tiers are worth keeping apart. Heteroscedastic or autocorrelated errors do not bias the line, but they make the reported standard errors — and therefore any claim of “significance” — untrustworthy. Constant variance and no autocorrelation repair the standard errors; normality (or a large sample) is what finally licenses reading a p -value or a confidence interval off a t -distribution.

Exercises

3.1. Implement OLS from the analytical formulas (no library fitting) and read off β_0 and β_1 for rent on surface over the training set. Confirm that the slope equals $\text{Cov}(x, y) / \text{Var}(x)$. *Check: $\beta_0 \approx 264.6$, $\beta_1 \approx 12.0$, and the slope matches the covariance-over-variance value.*

3.2. Interpret the slope and intercept from the previous exercise in the units of the problem, and state which assumptions buy unbiasedness and which buy valid inference. *Check: the slope is euros of rent per square metre; unbiasedness needs mean-independent errors, while valid t/F inference additionally needs constant variance, no autocorrelation, and — in a small sample — normality.*

4. Baselines and evaluation metrics

We have a fitted line. Is it any good? Two questions hide here — **is it better than doing nothing?** and **how wrong is it, in numbers I can act on?** — and each needs its own tool.

4.1. The baseline: the minimum a model must beat

Definition 11. A **baseline** is the simplest defensible predictor, used as the minimum a model must beat. For regression the standard baseline is the **mean predictor**, which ignores the features and predicts the mean training label for every example. A model that does not beat the baseline has no value.

Note **training** label: like any model, the baseline is fitted on the training set (here, one number — its mean) and then judged on held-out data. It is a one-liner:

```
baseline = y_tr.mean()          # the one number the baseline always predicts
print(round(baseline, 2))
```

```
1110.38
```

4.2. Metrics: how wrong, in what units

Definition 12. For predictions $\hat{y}^{(i)}$ against labels $y^{(i)}$ over n examples:

$$\text{MAE} = \frac{1}{n} \sum_{i=1}^n |y^{(i)} - \hat{y}^{(i)}|, \quad \text{MSE} = \frac{1}{n} \sum_{i=1}^n (y^{(i)} - \hat{y}^{(i)})^2, \quad \text{RMSE} = \sqrt{\text{MSE}}.$$

$$R^2 = 1 - \frac{\sum_{i=1}^n (y^{(i)} - \hat{y}^{(i)})^2}{\sum_{i=1}^n (y^{(i)} - \bar{y})^2}.$$

Read these in plain words. **MAE** (mean absolute error) is the average miss in euros; robust and easy to explain. **MSE** (mean squared error) squares each miss, so one €300 error counts far more than three €100 errors; its units are euros-squared, hard to interpret. **RMSE** takes the square root to return to euros while keeping that heavy penalty on large errors. R^2 rescales the error against the variance of the labels: it is the fraction of the target's variance the model explains. $R^2 = 1$ is a perfect fit; $R^2 = 0$ is exactly the score of a predictor that always outputs **the evaluation set's own mean**; and a negative R^2 means you did **worse** than that. Each is a few lines of numpy:

```
def scores(y, y_hat):
    err = y - y_hat
    mse = (err ** 2).mean()
    return {
```

```

    "MAE": float(np.abs(err).mean()),
    "MSE": float(mse),
    "RMSE": float(mse ** 0.5),
    "R2": float(1 - (err ** 2).sum() / ((y - y.mean()) ** 2).sum()),
}

x_te = X_te[:, 0]
model_pred = beta0 + beta1 * x_te # our fitted line
baseline_pred = np.full_like(y_te, baseline, dtype=float)

print("model: ", {k: round(v, 2) for k, v in scores(y_te, model_pred).items()})
print("baseline:", {k: round(v, 2) for k, v in scores(y_te, baseline_pred).items()})

```

```

model:    {'MAE': 72.38, 'MSE': 7865.21, 'RMSE': 88.69, 'R2': 0.94}
baseline: {'MAE': 307.52, 'MSE': 126427.8, 'RMSE': 355.57, 'R2': -0.01}

```

The verdict is decisive: the model's typical miss is about €89 against the baseline's €356, and it explains 94% of the variance in rent while the baseline explains essentially none. (The baseline's R^2 is -0.01 , just below zero rather than exactly zero, because it predicts the **training** mean, not the test set's own mean; a constant that is not the evaluation set's mean scores just under zero.) The line has learned real signal from surface.

Example — letting the business choose the metric. A landlord who loses money in proportion to each euro of misquoted rent cares about **MAE** — every euro counts the same. A pricing team that can absorb small errors but is hurt badly by the occasional wildly wrong quote should optimise **RMSE**, which punishes big misses hardest. The metric is a business decision, not a default.

Pitfall. Always report the metric **on held-out data**. Every score above is computed on y_{te} , never on the training set. MAE or R^2 measured on the training set flatters the model — it can look excellent there and fail on new apartments.

Exercises

4.1. Compute the mean-predictor baseline — the single number it always predicts — and its MAE on the test set. *Check: the baseline predicts the training-label mean, about €1110, for every flat.*

4.2. Implement the mean-predictor baseline (fitted on the training set) and compare a fitted model against it on the test set. Explain why a model that fails to beat the baseline has no value. *Check: the baseline's test R^2 is about 0, the fitted line's about 0.94.*

4.3. Compute MAE, MSE, RMSE and R^2 for the simple model on the test set. Given two contrasting business contexts, select and justify a different metric for each. *Check: test RMSE $\approx$ €89, MAE $\approx$ €72.*

5. Gradient descent

OLS handed us the optimum in one formula. That is a luxury of linear regression with squared error; most models have no closed form, and even linear regression on millions of rows may be cheaper to solve iteratively. The general-purpose optimiser that underlies almost all of modern machine learning is **gradient descent**, and it is worth implementing here, on a problem where we can check it against the exact answer.

The idea is physical. The loss $L(\beta)$ is a landscape over the coefficients; training means walking downhill to its lowest point. At any position the gradient ∇L points in the direction of steepest **increase**, so stepping in the opposite direction goes downhill fastest.

Definition 13. **Gradient descent** minimises a loss $L(\beta)$ by repeatedly stepping against the gradient:

$$\beta \leftarrow \beta - \alpha \nabla L(\beta),$$

where $\alpha > 0$ is the **learning rate**. The steps repeat until the loss stops decreasing meaningfully — **convergence**.

The loss we descend on here is the **mean squared error** — the RSS of Chapter 3 divided by the number of training rows n . Dividing a cost by the positive constant n cannot move where it bottoms out, so MSE and RSS share exactly the same minimiser: gradient descent on the MSE must land on the very OLS line, which is precisely what lets us check its answer against the closed form. (Carrying the $1/n$ also keeps the gradient's size from growing merely because we summed over more rows.) For that MSE loss the two partial derivatives are

$$\frac{\partial L}{\partial \beta_0} = -\frac{2}{n} \sum_{i=1}^n (y^{(i)} - \hat{y}^{(i)}), \quad \frac{\partial L}{\partial \beta_1} = -\frac{2}{n} \sum_{i=1}^n (y^{(i)} - \hat{y}^{(i)})x^{(i)},$$

which is all you need to code the loop: predict, compute these two numbers, nudge β_0 and β_1 , repeat.

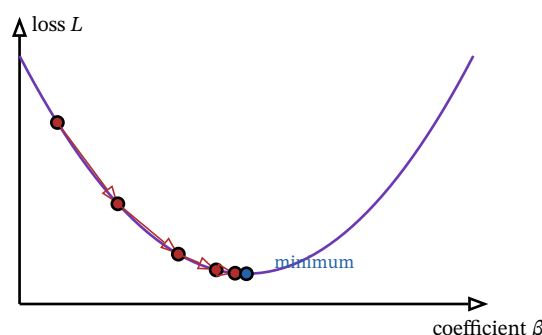


Figure 5: Gradient descent on a loss bowl. From a starting guess (left) each step moves against the gradient, downhill toward the minimum. Steps shrink as the slope flattens near the bottom — the algorithm naturally slows as it converges.

One practical point before we code: gradient descent behaves far better when features are on comparable scales, so we standardise surface (subtract its mean, divide by its standard deviation) before descending, then convert the coefficients back to the original euros-per-m² units at the end.

```

mu, sd = x_tr.mean(), x_tr.std()
xs = (x_tr - mu) / sd                                # standardised feature

def gradient_descent(xs, y, alpha, n_iters):
    w0, w1 = 0.0, 0.0                                # start at the origin
    n = len(y)
    history = []
    for _ in range(n_iters):
        y_hat = w0 + w1 * xs
        err = y_hat - y
        g0 = (2 / n) * err.sum()
        g1 = (2 / n) * (err * xs).sum()
        w0 -= alpha * g0                                # step against the gradient
        w1 -= alpha * g1
        history.append((err ** 2).mean())                # record the loss
    return w0, w1, history

w0, w1, hist = gradient_descent(xs, y_tr, alpha=0.1, n_iters=200)
beta1_gd = w1 / sd                                    # undo the standardisation
beta0_gd = w0 - w1 * mu / sd
print(round(beta0_gd, 2), round(beta1_gd, 3))
print("loss, first 6 iters:", [round(h) for h in hist[:6]])
print("loss at convergence:", round(hist[-1]))

```

```

264.61 12.0
loss, first 6 iters: [1357893, 872019, 561060, 362046, 234677, 153161]
loss at convergence: 8244

```

Gradient descent lands on €264.61 and €12.0 — the **same** line OLS gave us in one step. The printed loss history makes the mechanism concrete: it starts near 1.36 million, falls about ninefold over the first six iterations, and keeps shrinking — slowing as it nears the bottom — until it flattens near 8244 after a couple of hundred iterations. The learning rate is what governs this descent.

Definition 14. The **learning rate** α sets the step size and is the single setting that most often makes or breaks training. Too small, and convergence crawls, taking thousands of steps. Too large, and the steps overshoot the minimum, bounce up the far wall, and the loss **diverges** — growing each iteration instead of shrinking.

You can see all three regimes by rerunning the loop with different α (loss after 50 iterations):

```

for a in [0.001, 0.1, 1.05]:
    _, _, h = gradient_descent(xs, y_tr, alpha=a, n_iters=50)
    print(a, round(h[-1]))

```

```

0.001 1117454
0.1 8244
1.05 15371077849

```

The three rows are the three regimes. At $\alpha = 0.001$ the loss has barely moved from its starting value after 50 steps — far too small. At $\alpha = 0.1$ it has settled at 8244 — a good rate. At $\alpha = 1.05$ it has exploded past 10^{10} — too large, the steps overshoot and grow without bound.

Pitfall. A loss that grows every iteration is the clear sign of a learning rate that is too large — not a bug in your gradient. Before re-deriving the derivative, divide α by ten and rerun. Conversely, a loss that barely moves after many steps usually means α is too small — or that you forgot to scale the features.

Exercises

5.1. Standardise the surface feature (subtract its mean, divide by its standard deviation) and run the gradient-descent loop once at $\alpha = 0.1$, printing the loss every few iterations. *Check: the loss decreases every iteration and flattens near 8244.*

5.2. Implement gradient descent to fit the same regression on the standardised feature. Plot the loss against iteration for three learning rates: one too small, one well chosen, and one large enough to diverge. Relate each curve to the value of α . *Check: a well-chosen rate recovers the OLS line ($\beta_1 \approx 12.0$); the diverging rate's loss grows every iteration.*

6. Underfitting and overfitting

Linear regression assumes a straight-line relationship. But rent per square metre tends to fall for larger flats — the rent-versus-surface curve **bends**. To see this clearly we zoom in on a small, deliberately curved slice of the rental market: fourteen apartments whose rent rises steeply for small flats then flattens off. We can give a linear model the power to bend by feeding it powers of the feature, $x, x^2, \dots, x^d$ — **polynomial regression** of degree d . That single knob, the degree, lets us watch the central tension of all machine learning play out.

Example — three fits to the same curved, noisy data. Fit degree 1 (a line) and it is too rigid to follow the bend — it misses in a systematic way (**underfitting**). Fit degree 10 and the curve threads through **every** point, weaving between them and rising steeply near the edges (**overfitting**). Fit degree 3 and it follows the real bend while ignoring the jitter. Only the middle fit will predict new apartments well.

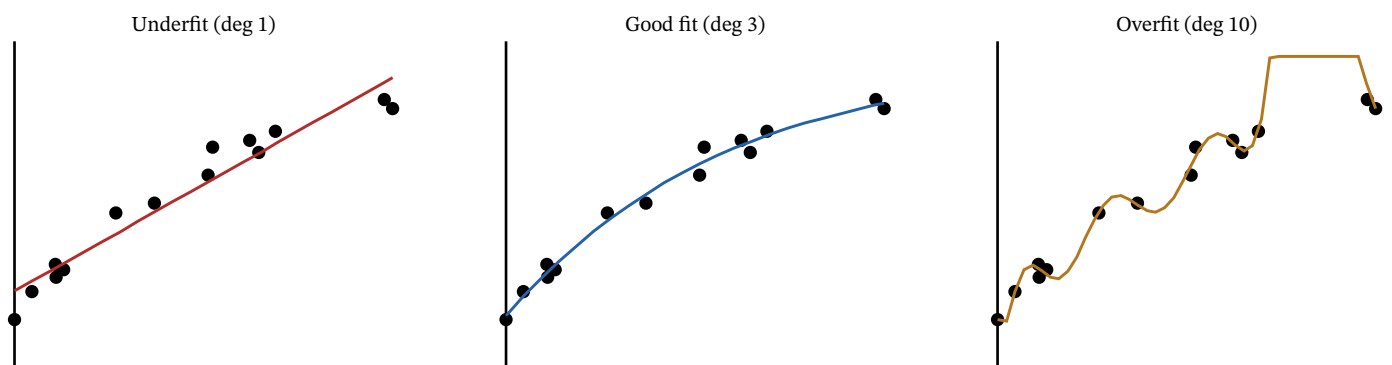


Figure 6: The same fourteen curved, noisy points, three model complexities. Left: too rigid, missing the bend (high error everywhere). Right: so flexible it interpolates the noise — it wiggles between points and swings sharply at the edge (perfect on these points, useless on new ones). Centre: captures the signal, ignores the noise.

Definition 15. A model **underfits** when it is too simple to capture the structure in the data: it has high error on **both** the training set and unseen data. A model **overfits** when it is so flexible it fits the noise in the training set: **low** training error but **high** error on unseen data. The gap between training and unseen error is the fingerprint of overfitting.

6.1. The validation curve: which complexity?

The fourteen curved apartments are their own reproducible sample; we hold out four of them as a validation set:

```
rng = np.random.default_rng(11)
cx = np.sort(rng.uniform(20, 120, 14))           # surface, m2
cy = (300 + 1100*np.sin(cx/75) + rng.normal(0, 45, 14)).round() # rent
val = [2, 6, 9, 12]                             # 4 held-out indices
tr = [i for i in range(14) if i not in val]       # 10 training indices
```

You diagnose which regime you are in by fitting each degree, then measuring error on the training points **and** on the held-out validation points (columns: degree, training RMSE, validation RMSE):

```
for deg in range(1, 9):
    coef = np.polyfit(cx[tr], cy[tr], deg)          # fit on the training points
    rmse = lambda xx, yy: ((yy - np.polyval(coef, xx)) ** 2).mean() ** 0.5
    print(deg, round(rmse(cx[tr], cy[tr])), round(rmse(cx[val], cy[val])))
```

```
1 66 51
2 28 30
3 27 31
4 27 34
5 26 35
6 22 141
7 22 101
8 12 5442
```

Training error keeps falling as the degree rises, but validation error falls, bottoms out around degree 2–3, then climbs steeply — underfitting at degree 1 (both errors high), the sweet spot at degree 2–3, and by degree 6–8 the validation error explodes while training error stays tiny (the signature of overfitting). This plot, of training and validation error against model complexity, is useful enough to have a name.

Definition 16. A **validation curve** plots training and validation error against **model complexity** (here, the polynomial degree). Underfitting sits at low complexity (both errors high and close); overfitting at high complexity (a widening gap — low training error, high validation error); and the best complexity is where the validation error is lowest.

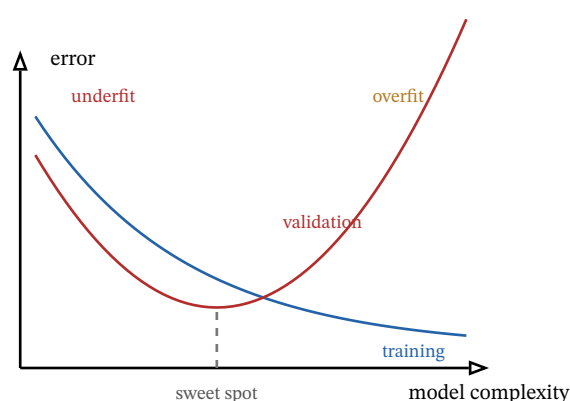


Figure 7: The validation curve, the shape traced by the numbers above. Training error (blue) always falls as the model grows more flexible. Validation error (red) falls, bottoms out, then rises as the model starts fitting noise. Choose the complexity at the bottom of the validation curve.

6.2. The learning curve: does more data help?

The validation curve answers **which complexity to choose**. A second, equally useful plot answers a different question — **would collecting more data help?** — by fixing the model and varying the amount of training data instead.

Definition 17. A **learning curve** plots training and validation error against the **training-set size**. An **underfitting** model's two curves meet at a **high** error and stay there: more data does not help, because the model is too simple to do better. An **overfitting** model shows a **large gap** — low training error, high validation error — that **narrows** as the training set grows: here more data does help, because more examples are harder to memorise than few.

Building one uses the same two measurements as before — training error and validation error — recorded as the training set grows. A learning curve varies the amount of data, so we need more of it than the fourteen-point sample above; we draw three hundred apartments from the same bending rule. For each training-set size we fit on that many points and score one fixed validation set, averaging over several random draws so that a single unlucky subset does not dominate (columns: degree, training-set size, training RMSE, validation RMSE):

```
rng = np.random.default_rng(7)
gx = np.sort(rng.uniform(20, 120, 300))          # surface, m2
gy = (300 + 1100*np.sin(gx/75) + rng.normal(0, 60, 300)).round() # rent

perm = np.random.default_rng(0).permutation(300)
val_i, pool = perm[:100], perm[100:]           # a fixed validation set; the rest a pool

def learning_curve(deg, sizes, repeats=30, seed=1):
    r = np.random.default_rng(seed)
    for m in sizes:
        tr, va = [], []
        for _ in range(repeats):                # average over random
            subsets of size m
            s = pool[r.choice(len(pool), m, replace=False)]
            mu, sd = gx[s].mean(), gx[s].std()    # standardise, as for
            gradient descent
            coef = np.polyfit((gx[s] - mu) / sd, gy[s], deg)
            pred = lambda xx: np.polyval(coef, (xx - mu) / sd)
            rmse = lambda xx, yy: ((yy - pred(xx)) ** 2).mean() ** 0.5
            tr.append(rmse(gx[s], gy[s])); va.append(rmse(gx[val_i], gy[val_i]))
            print(deg, m, round(np.mean(tr)), round(np.mean(va)))

for deg in (1, 5):
    learning_curve(deg, [15, 30, 60, 120, 200])
```

```
1 15 64 83
1 30 70 78
1 60 72 77
1 120 73 76
1 200 72 76
5 15 40 189
5 30 50 64
5 60 52 59
5 120 53 57
5 200 53 57
```

The two curves tell opposite stories. The degree-1 model is too rigid for the bend, so its two errors converge **together and high** — around €74 — and no amount of extra data pulls that floor down: this

is the mark of underfitting, and the cure is a more flexible model, not more data. The degree-5 model starts with a **wide gap** — it fits fifteen points almost perfectly (training error €40) while predicting the validation set poorly (€189) — but the gap shrinks steadily as data grows, until at 200 points the two errors nearly meet at a **lower** value (about €57) than the rigid model ever reaches: this is the mark of overfitting, and here more data genuinely helps.

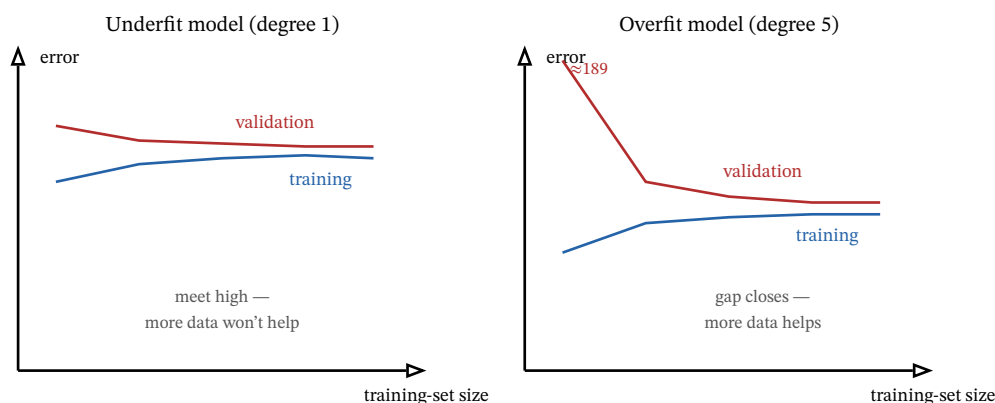


Figure 8: Two learning curves, drawn on the same scale. Left, an underfitting (degree-1) model: training and validation error meet at a high floor, and more data cannot lower it. Right, an overfitting (degree-5) model: a wide gap between low training error and high validation error narrows as the training set grows, converging to a lower error than the rigid model reaches. Two curves meeting high is the mark of underfitting; a gap that closes as data grows is the mark of overfitting.

Exercises

6.1. On the fourteen-point curved sample, fit a degree-1 and a degree-8 polynomial and compute each one's training and validation RMSE. Say which underfits and which overfits, and how the two errors tell you so. *Check: degree 1's two errors are both high and comparable; degree 8's training error is tiny while its validation error explodes into the thousands.*

6.2. Fit polynomials of degree 1 through 12 to a small, curved sample and plot a validation curve (training and validation error against degree); identify the underfitting and overfitting regimes and pick the degree where validation error is lowest. Then, on a larger curved sample, plot a learning curve (training and validation error against training-set size) for a degree-1 model and a degree-5 model, and read the regime off the shape of each. *Check: on the validation curve, training error falls monotonically while validation error bottoms out around degree 2–3 then climbs; on the learning curves, the degree-1 model's two errors meet high (around €75) whereas the degree-5 model's wide gap closes toward a lower error (around €57) as the training set grows.*

7. Regularization

The overfitting cure need not be a coarser model. Overfit linear models reveal themselves through **large, opposing coefficients** that let the curve swing wildly. **Regularization** tames this by adding the size of the coefficients to the loss, so the optimiser must trade fit against simplicity.

Definition 18. Regularization adds a penalty on coefficient size to the loss. **L2 regularization** (ridge) penalises the sum of squared coefficients; **L1 regularization** (lasso) penalises the sum of absolute coefficients:

$$L_{\text{ridge}} = \text{RSS} + \lambda \sum_{j=1}^p \beta_j^2, \quad L_{\text{lasso}} = \text{RSS} + \lambda \sum_{j=1}^p |\beta_j|.$$

The strength $\lambda \geq 0$ controls the trade-off: $\lambda = 0$ recovers plain OLS, and larger λ shrinks coefficients harder.

The two penalties differ in a consequential way: **L2 shrinks the coefficient vector smoothly and rarely sends any coefficient to exactly zero**, while **L1 drives some coefficients exactly to zero**, switching those features off entirely — it performs automatic feature selection.

The reason is geometric, and seeing it takes one bridge from the penalty above. Adding a penalty $\lambda \cdot (\text{size})$ to the loss has an equivalent **constrained** form: minimise the RSS while keeping the coefficients inside a fixed budget — $\sum_j \beta_j^2 \leq t$ for ridge, $\sum_j |\beta_j| \leq t$ for lasso — where a larger λ matches a tighter budget t . That budget is a **region** in coefficient space: a circle for ridge, a diamond for lasso. The fitted coefficients sit where the RSS contours, growing outward from the unconstrained OLS point, first touch that region. L1's diamond has sharp corners on the axes, so the first touch tends to land at a corner — where one coordinate is exactly zero; L2's smooth circle has no corners, so its touch point almost always keeps every coordinate nonzero.

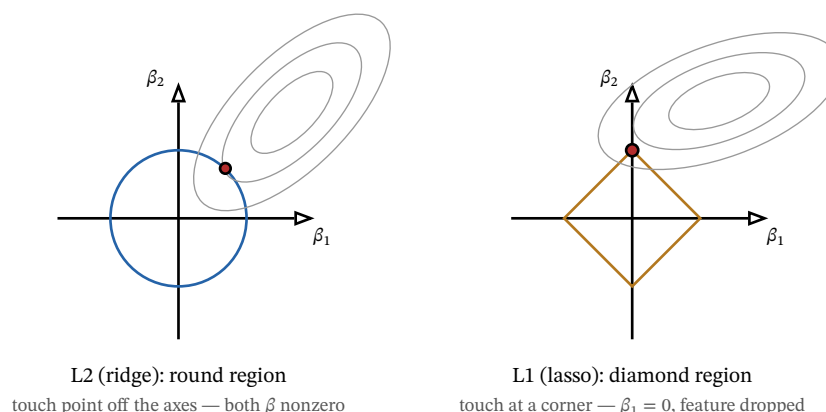


Figure 9: Why L1 selects features and L2 does not. The grey ellipses are loss contours; the coloured shape is the region the penalty allows. The fit is the first contour to touch that region. L2's circle is touched off the axes (both coefficients survive); L1's diamond is usually touched at a corner, forcing a coefficient to exactly zero.

We can watch this happen. Add three pure-noise features to our data, standardise so the penalty treats every coefficient on the same footing, and compare plain OLS, ridge and lasso:

```

from sklearn.linear_model import LinearRegression, Ridge, Lasso
from sklearn.preprocessing import StandardScaler

rng = np.random.default_rng(1)
noise_feats = rng.normal(0, 1, (200, 3))          # 3 irrelevant columns
Xreg = np.column_stack([surface, rooms, balcony, noise_feats])
Xr_tr, Xr_te, yr_tr, yr_te = train_test_split(Xreg, rent, test_size=0.3,
random_state=0)

scaler = StandardScaler().fit(Xr_tr)              # fit on TRAIN only
Xr_tr = scaler.transform(Xr_tr)

# feature order: surface  rooms  balcony  noise1  noise2  noise3
coefs = {}
for name, model in [("OLS ", LinearRegression()),
                    ("Ridge", Ridge(alpha=10)),
                    ("Lasso", Lasso(alpha=20, max_iter=10000))]:
    model.fit(Xr_tr, yr_tr)
    coefs[name.strip()] = model.coef_
    print(name, model.coef_.round(1))

```

```

OLS    [301.6  43.3  56.    0.4  10.4 -9.3]
Ridge  [233.8  98.1  53.8   4.8  13.4 -7.7]
Lasso  [296.3  29.   36.7   0.    0.   -0. ]

```

The six coefficients are, in order, surface, rooms, balcony, then the three pure-noise features.

Lasso set all three noise coefficients to **exactly** zero, keeping only the genuine features — automatic feature selection. Ridge kept every coefficient nonzero and pulled the overall coefficient vector in: its length falls from 310.1 (OLS) to 259.7. But shrinking the **vector** is not the same as shrinking each coefficient. Surface and rooms are correlated here — bigger flats have more rooms, as we saw in Chapter 3 — and ridge shifted weight off surface (301.6 → 233.8) onto the collinear rooms (43.3 → 98.1), whose coefficient more than doubled. Ridge **redistributes** weight across correlated features rather than driving every coefficient down one by one — the very behaviour that makes it the steadier choice when features move together. (Because the features are standardised, these coefficients are on a common “per standard deviation” scale, which is what makes the penalty fair and the magnitudes comparable.)

So: **prefer L1 (lasso)** when you suspect many features are irrelevant and want a sparse, interpretable model that names the few that matter; **prefer L2 (ridge)** when you believe most features contribute a little and you only want to keep their magnitudes under control — especially when features are correlated, where ridge is the steadier choice.

Exercises

7.1. Standardise the six features and fit lasso (L1) once. List which coefficients it sets to exactly zero and which it keeps. *Check: the three noise-feature coefficients are exactly 0; the three genuine features stay nonzero.*

7.2. Apply L1 (lasso) and L2 (ridge) regularization to the standardised multi-feature data (with a few added noise features). Report which coefficients each drives toward or exactly to zero, and give a situation where each penalty is the better choice. *Check: lasso sets the noise-feature*

coefficients to exactly 0; ridge keeps every coefficient nonzero, shrinking the vector overall rather than each coefficient one by one.

8. Cross-validation

The validation set we carved out has a weakness: its score depends on **which** apartments happened to land in it. An unlucky split — all the expensive flats in validation — gives a pessimistic, noisy estimate. With small datasets this randomness can dominate.

Example — the same model, five different scores. Score the **identical** simple-regression model on five different validation folds and you get RMSE values of €85, €106, €89, €85 and €88. Which is “the” performance? None alone; the honest answer is their average, €91, and the spread (from €85 to €106) warns you how uncertain that average is — one unlucky fold reads €106. That is precisely what cross-validation computes, and those are the real five numbers from the by-hand code below.

Definition 19. k-fold cross-validation splits the data into k equal folds. In turn, each fold is held out as the validation set while the model is trained on the other $k - 1$ folds; this yields k scores, which are averaged. Every example is used for validation exactly once and for training $k - 1$ times, so the estimate uses all the data and is far more stable than a single split.

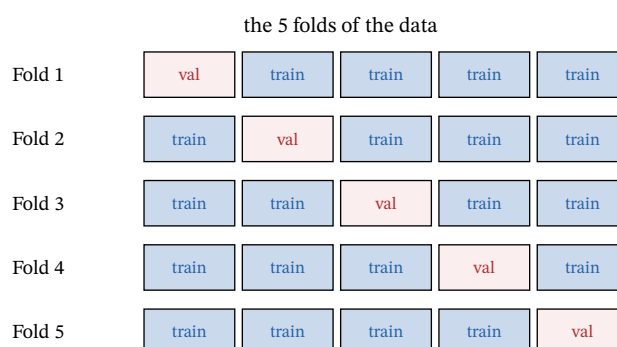


Figure 10: 5-fold cross-validation. Each row is one round: a different fifth of the data (red) is held out for validation while the rest (blue) trains the model. The five validation scores are averaged into one robust estimate, and every example is validated exactly once.

The definition **is** the algorithm, so we can implement it by hand before reaching for a library. Shuffle the row indices, cut them into k blocks, and loop:

```
def kfold_rmse(X, y, k=5, seed=0):
    idx = np.random.default_rng(seed).permutation(len(y))
    folds = np.array_split(idx, k)
    rmse = []
    for i in range(k):
        val_i = folds[i]
        tr_i = np.concatenate([folds[j] for j in range(k) if j != i])
        b0, b1 = ols_simple(X[tr_i], y[tr_i]) # train on k-1 folds
        pred = b0 + b1 * X[val_i] # score the held-out fold
        rmse.append(((y[val_i] - pred) ** 2).mean() ** 0.5)
    return np.array(rmse)

r = kfold_rmse(surface, rent, k=5)
print(r.round(1), "mean", round(r.mean(), 1), "std", round(r.std(), 1))
```

```
[ 84.7 105.6  88.7  85.4  88.4] mean 90.5 std 7.7
```

Once the mechanism is clear, scikit-learn does the same in one call — and this is what you use in practice (it shuffles differently, so the folds differ, but the average agrees):

```
from sklearn.model_selection import KFold, cross_val_score

kf = KFold(n_splits=5, shuffle=True, random_state=0)
cv_scores = cross_val_score(LinearRegression(), surface.reshape(-1, 1), rent,
                             cv=kf, scoring="neg_root_mean_squared_error")
print((-cv_scores).round(1), "mean", round((-cv_scores).mean(), 1))
```

```
[83.8 95.9 88.9 96.8 88.1] mean 90.7
```

Cross-validation costs k times the training, but on the small-to-medium datasets typical of an intro course that cost is trivial and the gain in reliability is large. It is the default way to estimate performance and, as we see next, to tune.

Exercises

8.1. Shuffle the row indices and cut them into 5 folds with `np.array_split`, then print each fold's size and confirm the folds together cover every row exactly once. *Check: five folds whose sizes sum to the number of rows, with no index repeated or missing.*

8.2. Implement k -fold cross-validation by hand. Compare its averaged estimate, and the spread of its per-fold scores, against the estimate from a single train/test split. *Check: 5-fold RMSE for the simple model averages about €91, with individual folds ranging from roughly €85 to €106.*

9. Hyperparameters

We have quietly introduced several knobs that are **not** learned from the data by fitting: the polynomial degree d , the regularization strength λ , the learning rate α . These are different in kind from the coefficients.

Definition 20. A **parameter** is learned from the data during training (the coefficients β). A **hyperparameter** is a setting fixed **before** training that controls how training happens or how complex the model may be (λ, d, α). Parameters are **learned**; hyperparameters are **chosen**.

You choose a hyperparameter by trying candidate values, training a model for each, and comparing them — but comparing them **by cross-validation** (or on the validation set), never on the test set. A degree-12 polynomial is flexible enough to overfit the curved sample wildly, even more than the degree-10 fit of Figure 6; regularization can rescue even that flexible a model, if we pick the right λ . We reuse `cx`, `cy` from the overfitting chapter:

```
from sklearn.preprocessing import PolynomialFeatures
from sklearn.pipeline import make_pipeline

for lam in [0.00001, 0.001, 0.01, 1, 100]:
    model = make_pipeline(PolynomialFeatures(12), StandardScaler(), Ridge(alpha=lam))
    cv = -cross_val_score(model, cx.reshape(-1, 1), cy, cv=kf,
                          scoring="neg_root_mean_squared_error").mean()
    print(lam, round(cv, 1))
```

```
1e-05 76.9
0.001 42.8
0.01 44.5
1 56.4
100 173.8
```

The cross-validated error is high with almost no penalty (the polynomial overfits), bottoms out around $\lambda = 0.001$, and climbs again once the penalty is so strong the model can no longer bend — the same U-shape as the validation curve, now traced by the hyperparameter. You keep $\lambda = 0.001$, and **only then**, with it fixed, unseal the test set for one final measurement.

Pitfall. Choosing a hyperparameter by its **test-set** score leaks the test set into the model: the test set has now influenced a modelling decision, so its score is no longer an honest estimate of performance on truly unseen data — it is optimistically biased. The validation set (or cross-validation) exists precisely so the test set can stay sealed. Tune on validation; report on test, once.

Exercises

9.1. List every hyperparameter met so far — the polynomial degree d , the regularization strength λ , the learning rate α — and, for each, say whether it is learned or chosen and what it controls. *Check: all three are chosen before training, not learned from the data by fitting.*

9.2. Tune the regularization strength λ by cross-validation. Then evaluate once on the test set. Explain precisely why tuning λ on the test set would invalidate the final number. *Check: cross-validated error is a U-shape in λ — high at very small λ , lowest at an interior value, high again when λ is very large.*

10. From simple to multiple regression

Surface alone ignores rooms and balcony, which clearly matter too. Multiple regression uses them all at once.

Definition 21. Multiple linear regression predicts from several features:

$$\hat{y} = \beta_0 + \beta_1 x_1 + \beta_2 x_2 + \dots + \beta_p x_p.$$

Each coefficient β_j is the predicted change in the target for a one-unit increase in feature x_j **with all other features held fixed** — the “all else equal” clause is what makes multiple regression more than several simple ones stacked together.

Fitting all three features at once recovers coefficients in the ballpark of the rule we built the data with (€9.5 per m², €55 per room, €110 for a balcony) — as close as 120 noisy examples allow:

```
model = LinearRegression().fit(X_tr, y_tr)          # surface, rooms, balcony
print(model.coef_.round(2), round(model.intercept_, 1))
```

```
[ 10.44  41.59 117.35] 218.6
```

Compare surface’s coefficient here, €10.44, with the €12.0 we got from surface **alone** in Chapter 3. The simple-regression slope was inflated because it silently carried the effect of rooms (bigger flats have more rooms); once rooms is in the model explicitly, surface’s “all else equal” effect drops toward its true €9.5. That shift is the whole point of the clause.

That “all else equal” interpretation runs into trouble when features move together.

Example — when two features carry the same information. Add a second feature, **surface in ft²**. In principle it is just **surface in m²** times 10.7639, but a real dataset never lines up that perfectly — a second survey rounds to the nearest square foot — so the two columns are **near-duplicates**, carrying essentially the same information twice. Fit the model on three bootstrap resamples of the data and the two surface coefficients change wildly, even flipping sign, because the model cannot decide how to divide one real effect between two near-identical features. Their **sum** (and the prediction) stays stable throughout.

Definition 22. Multicollinearity is strong correlation among features. It makes individual coefficients **unstable and uninterpretable** — small changes in the data swing them widely — because the data cannot separate the effect of one correlated feature from another. The model’s predictions can stay accurate even as its coefficients become meaningless.

A first diagnosis inspects the correlation matrix of the features — a pair correlating near 1.00 is the red flag — and watches for coefficients that are implausibly large, unstable across resamples, or carry the wrong sign. The correlation matrix is printed here as a four-by-four grid, in the column order surface, surface-in-ft², rooms, balcony:

```

rng = np.random.default_rng(3)
surface_ft2 = (surface * 10.7639 + rng.normal(0, 3, 200)).round() # a near-duplicate
Xdup = np.column_stack([surface, surface_ft2, rooms, balcony])

print(np.corrcoef(Xdup, rowvar=False).round(2))

boot = np.random.default_rng(0)
for t in range(3):
    idx = boot.integers(0, 200, 200) # a bootstrap resample
    m = LinearRegression().fit(Xdup[idx], rent[idx])
    print(t, round(m.coef_[0], 1), round(m.coef_[1], 1))

```

```

[[1.  1.  0.92 0.04]
 [1.  1.  0.92 0.04]
 [0.92 0.92 1.  0.02]
 [0.04 0.04 0.02 1.  ]]
0 15.3 -0.5
1 -19.1 2.8
2 -20.7 2.9

```

The correlation of 1.00 between the two surface columns is the diagnostic; the bootstrap coefficients confirm it. The first column's coefficient reads €15.3, then –€19.1, then –€20.7 across the three resamples — swinging widely and flipping sign — while the two columns' combined per-m² effect stays near €10 and the predictions barely move.

The correlation matrix has one blind spot: it compares features only two at a time. A feature can be a near-perfect combination of **several** others — and so ruin the coefficients just as badly — while correlating only weakly with each of them individually. The standard tool that catches this general case gives one number per feature.

Definition 23. The **variance inflation factor** (VIF) of a feature x_j measures how much collinearity inflates the variance of its coefficient. Regress x_j on **all the other** features and let R_j^2 be that regression's R^2 ; then

$$\text{VIF}_j = \frac{1}{1 - R_j^2}.$$

A VIF of 1 means x_j is uncorrelated with the rest. As a rule of thumb, a VIF above about 5 to 10 signals problematic multicollinearity. Because R_j^2 comes from regressing x_j on **every** other feature at once, VIF flags a feature collinear with a combination of others even when no single pairwise correlation is large — exactly what the correlation matrix misses.

```

def vif(X):
    vifs = []
    for j in range(X.shape[1]):
        others = np.delete(X, j, axis=1) # every feature except j
        r2 = LinearRegression().fit(others, X[:, j]).score(others, X[:, j])
        vifs.append(1 / (1 - r2)) # VIF_j = 1 / (1 - R^2_j)
    return np.array(vifs)

```

```
for name, v in zip(["surface", "surface_ft2", "rooms", "balcony"], vif(Xdup)):  
    print(f"{name:12} {v:8.1f}")
```

```
surface      9460.1  
surface_ft2  9454.4  
rooms        6.6  
balcony      1.0
```

The two surface columns score in the thousands — collinearity so severe the variance formula all but blows up — while balcony (1.0) is clean and rooms (6.6) sits just past the rule-of-thumb line, picking up its genuine 0.92 correlation with surface that the matrix already showed.

The remedy is usually to drop or combine the redundant features — or to use ridge regularization, which stabilises exactly this situation.

Exercises

10.1. Fit multiple regression on all three features and read off the three coefficients, interpreting each “all else equal” in the units of the problem. *Check: the coefficients are roughly €10 per m², €42 per room, and €117 for a balcony.*

10.2. Building on the previous exercise, note how surface’s coefficient changed from its simple-regression value. Then add surface-in-ft² as a near-duplicate feature, exhibit the resulting coefficient instability across resamples, and diagnose the multicollinearity from both the correlation matrix and the variance inflation factor. *Check: the duplicated pair correlates at 1.00, its coefficients swing wildly across resamples, and its VIF runs into the thousands while balcony’s stays near 1.*

11. Reproducible workflows: scikit-learn pipelines

Everything so far — split, preprocess, fit, tune, evaluate — must be applied in the **same order** and **fitted on the training data only**, every time. Do it by hand across notebook cells and a subtle leak creeps in: you scale using statistics computed from the whole dataset, including validation and test, and your scores quietly inflate. The software-engineering discipline you already have — reproducibility, clear contracts — has a direct instrument here.

Definition 24. A **scikit-learn pipeline** chains preprocessing steps and a model into one object. Calling `fit` runs each step's fit in order on the training data; calling `predict` applies each fitted transformation and then the model. Because every transformer is fitted only on the training portion, a pipeline **prevents data leakage** from validation or test data into preprocessing.

The pipeline is one object you can fit, cross-validate, and predict with — the scaler and the model travel together, so they can never be applied out of order or fitted on the wrong data:

```
from sklearn.pipeline import Pipeline

pipe = Pipeline([
    ("scaler", StandardScaler()),
    ("model", Ridge(alpha=10)),
])

# Inside cross-validation, the scaler is re-fitted on each fold's TRAINING part
# only, then applied to that fold's validation part — automatically, no leakage.
cv = -cross_val_score(pipe, X_tr, y_tr, cv=kf,
                      scoring="neg_root_mean_squared_error").mean()
print(round(cv, 1))

pipe.fit(X_tr, y_tr) # fit the whole chain on training data
print({k: round(v, 2) for k, v in scores(y_te, pipe.predict(X_te)).items()})
```

```
77.3
{'MAE': 61.45, 'MSE': 6152.75, 'RMSE': 78.44, 'R2': 0.95}
```

The cross-validated RMSE is 77.3, and the single final measurement on the sealed test set agrees with it (RMSE 78.44).

Searching for the best hyperparameter fits naturally on top: `GridSearchCV` wraps the pipeline, tries each λ by cross-validation, and refits the winner — the tuning of the previous chapter, in one object.

```
from sklearn.model_selection import GridSearchCV

grid = GridSearchCV(pipe, {"model__alpha": [0.1, 1, 10, 100]},
                    cv=kf, scoring="neg_root_mean_squared_error")
grid.fit(X_tr, y_tr)
print(grid.best_params_) # the lambda cross-validation preferred
```

```
{'model__alpha': 0.1}
```

A pipeline is the **reproducible form of the entire course**: the identical chain of steps runs during cross-validation, during hyperparameter search, and at final test time, so the number you report is the number a colleague re-running your code will get. That is where the ML project cycle and the engineering discipline meet — and where this course ends and every later model begins.

Exercises

11.1. Build a two-step pipeline (StandardScaler then Ridge) and fit it on the training data in a single fit call. *Check: one call scales and fits the model together, and the fitted pipeline can predict on the test set directly.*

11.2. Build a scikit-learn pipeline chaining a StandardScaler and a regularized regression, run it under cross-validation, and explain how the pipeline prevents preprocessing from leaking information out of the training folds. Then wrap it in GridSearchCV to tune λ . *Check: cross_val_score on the pipeline runs without ever fitting the scaler on validation data.*